

Final-Year Project Report

AI-Driven Observability and Root-Cause Analysis

Design and Implementation of the CIRES Observability Platform

Lina Laaraich

CIRES Technologies / Tanger Med — Digital Factory Observability Service

May 2026

Working draft, prepared as the structural and content basis for the final submitted report.

Abstract

This dissertation reports on the design and implementation of the *CIRES Observability Platform*, an end-to-end production observability stack augmented with an AI-driven Root-Cause Analysis (RCA) layer. The platform was built between February and May 2026 at CIRES Technologies (Tanger Med, Digital Factory Observability Service) across four planned sprints. Sprints 1–3 were completed during the project period; Sprint 4 is documented as scoped future work.

The work addresses an operational problem surfaced during the discovery phase: incidents in the target production environment were being discovered *after the fact* — by clients, or during retrospective log review — rather than by the monitoring system itself. The platform’s response is a two-layer design: a conventional observability foundation (Prometheus, Loki, Jaeger, Grafana, OpenTelemetry) provisioned by Ansible on AWS, with a FastAPI-based AI triage service on top that consumes alerts via Grafana webhooks, gathers cross-pillar context through five Model-Context-Protocol (MCP) bridge servers, and produces structured root-cause analyses with a local Large Language Model (`qwen2.5:7b-instruct` on Ollama).

The implementation introduces several novel mechanisms to keep the LLM output operationally trustworthy: an exemplar library of fourteen canonical RCA archetypes injected as structural scaffolding before the evidence pack; an alert-keyed hallucination blocklist; a surface-only and hypothesis-menu validator pair; a confidence-clamp that strips suggested actions when trust signals are weak and emits read-only *diagnostic steps* instead; a single bounded-agency retry constrained to one MCP call; and a closed-loop feedback layer that turns operator overrides into routing signal. An *MCP-only data-access invariant* is enforced as a project-wide rule: every datum the LLM sees must arrive via an MCP bridge, never via direct in-process imports or direct database/Prometheus queries.

The dissertation presents the architecture, sprint-by-sprint implementation history, an in-depth case study of the `0b215ef3` incident that drove the hallucination-firewall epic, an evaluation against a 4-axis RCA quality rubric, and a catalogue of the twenty-three durable design decisions (D1–D23) recorded during construction. The final chapter scopes future work, including iterative agentic gather, curated retrieval-augmented generation fed from operator feedback, and a labelled chaos-test corpus for regression-grade quality measurement.

Keywords: observability, SRE, AIOps, root-cause analysis, MCP, Prometheus, Loki, Jaeger, Grafana, large language models, Ollama, qwen2.5, Ansible, Terraform, k3s.

Contents

Abstract	1
1 Introduction	5
1.1 Context and motivation	5
1.2 Problem statement	5
1.3 Aim and objectives	5
1.4 Scope and limitations	6
1.5 Structure of the dissertation	6
2 Background and Related Work	7
2.1 The three pillars of observability	7
2.2 SRE practice and the golden signals	7
2.3 Prometheus, Loki, Jaeger, Grafana	8
2.4 Drain3 and log-template clustering	8
2.5 Large language models in IT operations	8
2.6 The Model-Context-Protocol	9
2.7 Related platforms and prior art	9
3 Discovery and Requirements (Sprint 0)	11
3.1 Approach	11
3.2 Findings from interviews	11
3.3 Solution-space exploration	11
3.4 Functional requirements	12
3.5 Non-functional requirements	12
4 System Architecture	14
4.1 Layered architecture overview	14
4.2 Runtime topology	14
4.3 Data flow: alert to RCA	15
4.4 The MCP-only data-access invariant	15
5 Sprint 1 — Observability Foundation	16
5.1 Sprint goal and topology	16
5.2 Ansible role layout	16
5.3 Three-pillar telemetry	16
5.4 Grafana and initial alerting	16
5.5 Sprint 1 outcome	17
6 Sprint 2 — AWS Migration and AI RCA MVP	18
6.1 Sprint goal	18
6.2 Epic 1 — AWS Infrastructure	18
6.3 Epic 2 — AI RCA System MVP	18
6.4 Epic 3 — Kubernetes migration	19
6.5 Epic 4 — AI Triage Hardening	19
6.6 Epic 5 — UEBA-flavoured stories (post-sprint)	19
6.7 Sprint 2 outcome	19

7	Sprint 3 — Quality, Feedback, and Hallucination Firewall	20
7.1	Sprint shape	20
7.2	The 4-axis RCA quality rubric	20
7.3	The exemplar library (D17)	21
7.4	Adaptive thresholds (D18)	21
7.5	RCA prose philosophy (D19)	21
7.6	Closed-loop feedback (D20)	21
7.7	Recurrence gates (D21)	22
7.8	Per-service Drain3 template trees (D23)	22
7.9	The hallucination firewall epic (Epic 4)	22
7.10	The 20-day investigation window (2026-04-30 to 2026-05-19)	23
7.11	Documentation refresh	23
7.12	The bounded-agency retry	24
8	Case Study: the 0b215ef3 Incident	25
8.1	The alert	25
8.2	The bad RCA	25
8.3	Forensic analysis: three MCP-invariant violations	25
8.4	The tiered response	26
8.5	Lessons captured	26
9	Evaluation	27
9.1	Test-coverage progression	27
9.2	RCA quality on the 28-decision sample	27
9.3	Chaos-run outcomes	27
9.4	Audit-corpus consistency	28
9.5	Latency budget	28
9.6	Operational steady state	28
10	Design Decisions and Rationale	29
10.1	Three decisions in depth	30
10.1.1	D17 — Why exemplars work where retrieval did not	30
10.1.2	The MCP-only invariant as a project-wide rule	30
10.1.3	Bounded agency over full agentic gather	31
11	Future Work and Sprint 4 Roadmap	32
11.1	Sprint 4 candidate ranking	32
11.2	Tier 3 — iterative agentic gather	32
11.3	Curated RAG retrieval (US-3.3 / US-3.4)	33
11.4	Resilience tier (R-1 to R-5)	33
11.5	Dropped scope: cloud GPU migration	33
11.6	Beyond Sprint 4	34
12	Conclusion	35
A	Repository inventory	36
B	Glossary	37
C	Sprint chronology summary	39
	References	40

List of Figures

4.1	Layered architecture. Each layer consumes only the layer immediately below it; the MCP-only invariant is the rule that formalises this for the AI layer.	14
4.2	The ten-stage triage pipeline. Stages 6/6b/6c/6d together form the hallucination firewall.	15

List of Tables

6.1	Epic 5 stories carried past Sprint 2 close.	19
7.1	Investigation-window outcomes (2026-04-30 to 2026-05-19). Each candidate was scored on the 4-axis RCA quality rubric over a twenty-eight-decision sample. . .	24
9.1	Triage-service test-suite progression.	27
9.2	Baseline RCA quality on the 28-decision sample (higher is better). Actionable is the lowest score because of the deliberate strip-actions-on-clamp behaviour (S3-HF-01), which lowers the actionability score but raises operational trust. . .	27
10.1	Catalogue of durable design decisions D1–D23.	29
11.1	Sprint 4 candidates as of 2026-05-19.	32
A.1	Project repository layout (seven repositories).	36
C.1	Sprint chronology, condensed.	39

CHAPTER 1

Introduction

1.1 Context and motivation

CIRES Technologies is the digital factory of Tanger Med, the principal port complex of northern Morocco. The Digital Factory Observability Service is responsible for the operational health of the internal service catalogue that supports port logistics, customer integrations, and back-office workflows. At the start of this project, the service catalogue ran on a small fleet of on-premises virtual machines hosting a representative web stack (a Spring Boot back-end, a Kong API gateway, a MySQL data store, a React-based front-end); a parallel migration to AWS was already on the medium-term roadmap.

Internal monitoring at the start of the project was limited to ad-hoc log inspection and a small number of metric-threshold rules. Operationally, this produced a structurally reactive posture: outages were detected by client reports or during retrospective log review, not by the system itself. The asymmetry between the speed of failure (seconds to minutes) and the speed of human detection (hours to days) defined the operational gap that this project sets out to close.

1.2 Problem statement

Problem statement (derived from Sprint 0 field research)

The team needs a way to detect production problems early — before they reach a client, and without requiring a human to be reading logs at the right moment. Detection should also be *pre-investigated*: each alert should arrive with a candidate root cause, the supporting evidence, and a recommended remediation, so that the operator's first move is to validate or override rather than to start an investigation from scratch.

This formulation separates the problem into two distinct halves. The first half is conventional: stand up a real observability stack so the system emits actionable signals. The second half is novel: layer on top of that stack an automated reasoning system that turns alerts into structured, actionable RCAs without on-call human attention. The platform implements both halves and integrates them end-to-end.

1.3 Aim and objectives

Aim. Design, implement and validate a production-shaped observability platform that proactively detects, investigates and reports on incidents using an AI triage layer that runs over open-source infrastructure, with no dependency on hosted SaaS observability vendors.

Objectives.

1. Establish a three-pillar observability foundation (metrics, logs, traces) for the target web stack, deployable from clean infrastructure via a single Ansible playbook.
2. Migrate the workload from on-premises VMs onto a Terraform-managed AWS environment and run the application workload on Kubernetes (k3s) while keeping the monitoring stack

on Docker Compose for operational simplicity.

3. Implement an AI triage service that consumes Grafana webhooks and produces structured root-cause analyses with verdicts, evidence, and suggested remediation actions, persisted to a durable history store.
4. Bridge each telemetry source (Prometheus, Loki, Jaeger, triage history, deployment events) behind an MCP server, and enforce an MCP-only data-access invariant so that every datum the LLM consumes is provenanced through a bridge.
5. Develop and evaluate quality-control mechanisms that keep LLM output operationally trustworthy: exemplar injection, a hallucination blocklist, prose-shape validators, a confidence clamp, and a closed-loop operator feedback channel.
6. Document the system end-to-end at a level sufficient for the engineering team to operate, extend and audit it after the project's conclusion.

1.4 Scope and limitations

The project explicitly scopes *in* the components above and explicitly scopes *out* the following:

- Hosted SaaS observability vendors (Datadog, New Relic, Honeycomb). These were costed and surveyed in Sprint 0; the decision to use open-source infrastructure was driven by budget, data-sovereignty, and learning objectives.
- Production rollout to actual customer-facing systems. The scope is the representative web stack used as a development surrogate (a derivative of the `mukundmadhav/react-springboot-mysql` repository), deployed on dedicated infrastructure with synthetic load and chaos-injected failure.
- Multi-region high-availability for the observability stack itself. The platform runs in `us-east-1` (or local on-premises) and is not multi-AZ.
- A cloud GPU runner for the LLM layer. This was originally scoped as a Sprint-4 candidate and was dropped on cost grounds on 2026-05-19 in favour of a laptop-based runner that proved sufficient (see §11.5).

1.5 Structure of the dissertation

The dissertation proceeds as follows. Chapter 2 reviews the technical background: observability practice, SRE, log-template clustering, large language models in IT operations, and the Model-Context-Protocol specification. Chapter 3 covers the Sprint-0 discovery phase, including interview findings and the framing that produced the problem statement above. Chapter 4 presents the platform architecture in its current shape. Chapters 5–7 cover the three completed implementation sprints in chronological order. Chapter 8 provides an in-depth case study of the `0b215ef3` incident that drove the hallucination-firewall epic. Chapter 9 evaluates the platform against a four-axis RCA quality rubric, the chaos-injection harness, and the recurring static and live audit cycles. Chapter 10 catalogues the twenty-three durable design decisions (D1–D23) recorded during construction and discusses the three most consequential in depth. Chapter 11 scopes Sprint 4 and beyond, and chapter 12 concludes.

CHAPTER 2

Background and Related Work

2.1 The three pillars of observability

Modern observability practice identifies three primary telemetry pillars: *metrics*, *logs*, and *traces*. Metrics are numeric time-series of system state (CPU utilisation, request rate, error rate, etc.), typically scraped on a fixed interval and stored in a purpose-built time-series database such as Prometheus. Logs are discrete textual events emitted by services, ideally with structured fields, aggregated by a log database such as Grafana Loki. Traces are distributed records of a single request’s journey across services, carrying a trace identifier that links spans recorded by each hop into one structure for examination; the OpenTelemetry project provides the canonical wire format and Jaeger is a widely-used storage backend.

A fully observable system requires all three pillars and the ability to navigate between them: from an anomalous metric to the logs of the service whose metric is anomalous, and from those logs to the trace that carried the originating request. The CIRES platform implements this triangulation as a first-class concern; the AI triage service’s context-gathering step explicitly fans out to all three pillars and to a fourth, Drain3-based log-template clustering signal, before any LLM reasoning takes place.

2.2 SRE practice and the golden signals

Google’s Site Reliability Engineering literature (the *SRE Book* and *SRE Workbook*) identifies four *golden signals* that almost any user-facing service should be measured against: **latency**, **traffic**, **errors**, and **saturation**. The book also articulates two ideas that influenced this project’s alert taxonomy:

- The **symptoms vs. causes** distinction. Alerts should fire on user-observable symptoms (high latency, elevated error rate) rather than on underlying causes (high CPU, high memory). Cause-based alerts produce a flood of false positives in healthy systems; symptom-based alerts page the operator only when something is actually broken from the user’s viewpoint.
- The **error budget** framing. Service-Level Objectives (SLOs) define a tolerable error rate; the gap between target reliability and 100 % reliability is the error budget. A team that is in the red on its error budget should slow down feature development and invest in reliability; a team in the green has explicit permission to take more risk.

The CIRES platform does not at this stage formalise SLOs or error budgets, but the alert rule set is deliberately structured to lean symptomatic — **HighP95Latency** on the gateway, **HighKongP95Latency** per upstream, **ErrorRate** on the service tier — with cause-shaped alerts (CPU, memory, disk) treated as supplementary diagnostic signal rather than primary paging material.

2.3 Prometheus, Loki, Jaeger, Grafana

The platform’s foundational stack is the de-facto open-source combination for modern infrastructure observability:

- **Prometheus** for metrics. Pull-based scraping on a 15-s interval. Spring Boot exposes `/actuator/prometheus`, Kong exposes its statistics endpoint, and host-level `node_exporter` and `cAdvisor` fill in the gaps.
- **Loki** for logs. Pull-based ingestion via Promtail. Loki stores log streams indexed by labels rather than by full-text content, which keeps storage costs low and pairs cleanly with Prometheus’s label model.
- **Jaeger** for traces. OpenTelemetry Collector receives spans from the OTel Java agent injected into Spring Boot, and from the Kong OTel plugin at the gateway layer, and forwards them to Jaeger. Trace identifiers propagate end-to-end.
- **Grafana** for visualisation and alerting. Grafana unified alerting replaced Alertmanager early in the project (see decision D6 in chapter 10); a single `triage-webhook` contact point routes all alert events to the AI triage service.

2.4 Drain3 and log-template clustering

Real production logs contain a small number of recurring template strings and a large body of incidental variation (timestamps, identifiers, parameter values). The Drain3 algorithm (a tunable variant of Drain) ingests log lines, replaces the variable parts with placeholders, and clusters lines by the resulting template. Once the catalogue of templates has stabilised, novel templates become a strong operational signal: a log line whose template has never been seen before is, by construction, unusual.

The CIRES platform integrates Drain3 as a fourth signal alongside metrics, logs, and traces. A background poller reads the most recent log lines from Loki, feeds them through a per-service template miner, and emits a synthetic `Drain3AnomalyDetected` webhook into the triage service whenever the recent novelty rate exceeds a configurable threshold. The webhook payload carries the actual novel template strings and a sample of the underlying lines, so the LLM downstream sees not just a count but the evidence itself (decision D23 records the per-service split that resolved a class of false-positive issues introduced by an earlier global miner).

2.5 Large language models in IT operations

The use of large language models for IT-operations tasks is an active area sometimes labelled *AI Ops*. The state of the practice varies widely. Hosted observability vendors have shipped AI-flavoured features for several years (Datadog Watchdog, New Relic AI, Honeycomb’s BubbleUp, Komodor’s incident summarisation), each with a different scope: anomaly detection, alert summarisation, correlation across signals, incident reconstruction.

Three risks dominate when an LLM is placed in the operational path:

1. **Hallucination.** The model may confidently produce statements that are not entailed by the evidence pack — invented metric values, services that do not exist, `kubectl` commands for systems that are not deployed on Kubernetes.

2. **Surface-only reasoning.** The model may restate the alert in different words (“the alert indicates high latency” or “based on the observed value of 1.2 s”) without identifying a cause.
3. **Hypothesis menus.** Under uncertainty, the model may emit a list of possibilities (“this could be a slow query, a connection pool issue, or a garbage-collection pause”) rather than committing to one and naming the evidence for it.

A central engineering challenge of the project has been the design of mechanisms that detect and reject each of these failure modes without rejecting genuinely good RCAs. Chapter 7 and chapter 8 treat these mechanisms in depth.

2.6 The Model-Context-Protocol

The Model-Context-Protocol (MCP) is an open specification, introduced in 2024 by Anthropic and now adopted across a range of LLM tooling projects, for connecting LLM applications to external data sources and tools. An MCP *server* exposes a small number of tools (typically read-only) over a JSON-RPC transport; an MCP *client* discovers tools, invokes them, and feeds the results back into the model’s context.

The CIRES platform uses MCP not in its original “LLM directly calls tools” shape but in a more constrained pattern. Five MCP servers sit between the data sources and the triage service:

- `prometheus-mcp:8091` — instant and range queries against Prometheus.
- `loki-mcp:8092` — log queries against Loki.
- `jaeger-mcp:8093` — trace lookups against Jaeger, plus a `get_trace` tool for full-trace retrieval.
- `rca-history-mcp:8094` — read access to the persistent RCA history (SQLite, on the laptop runner).
- `deploy-events-mcp:8095` — deployment-event timeline from the application repositories.

In the default path, the triage service *itself* (not the LLM) fans out to these MCP servers during context gathering and assembles an evidence pack that is rendered into the prompt as a structured **## Pre-Gathered Context** section. The LLM is invoked once on this pre-gathered material. Only the bounded-agency retry path (chapter 7, §7.12) gives the LLM direct access to MCP tools, and only one such tool call is permitted per retry.

This shape is deliberate: it preserves the LLM’s strengths (structured-output generation, narrative synthesis) while keeping the operational paths (which queries to run, which evidence to gather) under deterministic control. The **MCP-only invariant** — “every datum the LLM sees must arrive via an MCP bridge” — is the project’s hallucination firewall in its most general statement.

2.7 Related platforms and prior art

- **Datadog Watchdog** performs anomaly detection across metrics and produces narrative incident summaries. It is a proprietary, closed system; the CIRES platform’s open-source construction allows the team to inspect and tune every layer.

- **New Relic Applied Intelligence** (*NRAI*) provides correlation and prioritisation across alerts. Its “incident intelligence” graph is a useful precedent for the Sprint-4 *incident correlator* (US-5.2) that the CIRES platform scopes as future work.
- **Honeycomb BubbleUp** surfaces feature-value distributions that distinguish a slow request population from a fast one — an effective bottom-up tool but one that requires Honeycomb’s specific high-cardinality data model.
- **Komodor** produces narrative incident timelines for Kubernetes workloads. The CIRES platform’s exemplar archetypes and trace-depth gather (planned via `get_trace`) cover a similar conceptual space.

The closest published precedent for the platform’s overall shape is the body of work around structured-output LLM use in support engineering (“LLM as RCA copilot” rather than “LLM as autonomous responder”). The novel contributions of this project are (a) the MCP-only invariant treated as a project-wide rule, (b) the exemplar library of fourteen canonical RCA archetypes injected as prompt scaffolding, and (c) the layered prose-shape validator and confidence-clamp mechanism that together comprise the hallucination firewall.

CHAPTER 3

Discovery and Requirements (Sprint 0)

3.1 Approach

The project ran a deliberate Sprint 0 between February and the first days of March 2026 before any infrastructure code was written. Two parallel tracks ran for the duration of this sprint:

1. **Monitoring-solution benchmarking.** A survey of the major open-source observability stacks, their feature matrices, deployment models, and integration footprint with the JVM / Spring Boot stack already in use at CIRES. SaaS alternatives (Datadog, New Relic, Honeycomb) were costed and ruled out for this engagement on a mix of budget and data-sovereignty grounds.
2. **Field research.** Structured conversations with the engineers and operators closest to the production system, focused on three questions: what wakes them up, how they first discover that something is broken, and what their first investigation step is when an alert lands.

A third stream, conducted in parallel, was a guided read-through of Google’s *SRE Book* and *SRE Workbook* with explicit translation into Tanger Med’s context: a small team without a formal on-call rotation, a mixed-deployment service catalogue, and operational visibility concentrated in a small number of senior engineers.

3.2 Findings from interviews

The single most consequential finding from the field research recurred in nearly every conversation. When asked how problems were first discovered, the modal answer was a variant of:

Recurring interview finding

“We find out things are broken because a client tells us, or because someone notices after the fact in a log review.”

Detection was lagging actual breakage by hours to days. The team’s reactive posture was structural, not a matter of effort: there was nothing in the system that drew attention to a problem on its own. This finding crystallised the problem statement given in §1 and shaped the entire architecture that followed.

A second finding, less central but still important, was that even when alerts *were* in place (e.g. for disk usage on certain servers), the operators routinely had to do non-trivial investigation work to understand what the alert meant in the current context. The project’s emphasis on *pre-investigated* alerts — arriving with a candidate RCA, evidence, and remediation — traces back directly to this second finding.

3.3 Solution-space exploration

Several candidate architectures were brainstormed and pressure-tested before settling on the eventual two-layer design:

1. **Traditional rules-only alerting.** Rejected — it already exists conceptually, addresses the *detection* half of the problem only, and leaves the *investigation* half to a human reading dashboards at the right moment.
2. **AI-summarisation layer over raw alerts.** Closer: addresses the investigation half partially, by giving the operator a narrative summary alongside the alert. Risky because the summary is purely descriptive (“CPU is high”) rather than diagnostic (“CPU is high because the cron job is running and missing its lock”); it does not actually reduce the operator’s investigation load.
3. **Full AI-triage layer that turns alerts into structured RCAs with verdicts and remediation suggestions.** Chosen. Addresses both halves: the observability layer detects, the triage layer pre-investigates. Risks recorded explicitly: hallucination, surface-only output, model-availability dependence on a self-hosted LLM, latency budget of the LLM call vs. operator expectations.

3.4 Functional requirements

1. **FR-1** The platform shall ingest metric, log, and trace telemetry from the target web stack with no source-code modification of the application services. Instrumentation shall be *external*: agents mounted at start time, plug-ins configured at the gateway layer.
2. **FR-2** The platform shall raise alerts on user-observable symptoms (latency, error rate, traffic) and on a set of cause- shaped supplementary signals (CPU, memory, disk, target-down, log anomalies).
3. **FR-3** For every alert that is not suppressed by the deduplication or recurrence-gate layers, the platform shall produce a structured RCA containing: verdict (escalate / dismiss / inconclusive), root-cause prose, suggested actions or read-only diagnostic steps, a list of cited evidence, a list of correlated alerts in the surrounding window, and a confidence value.
4. **FR-4** The RCA shall be persisted to a durable history store, exposed via a dashboard, and dispatched as an escalation email when the verdict so indicates.
5. **FR-5** Operator overrides on RCAs (“actually dismiss” / “actually escalate”) shall be captured and used as a routing signal for subsequent fires.
6. **FR-6** All data sources consumed by the LLM shall be accessed via an MCP bridge (the MCP-only invariant).

3.5 Non-functional requirements

1. **NFR-1 Reproducibility.** The entire stack shall be reproducible from clean infrastructure via a single Ansible playbook (Sprint 1) or a Terraform + Ansible pair (Sprint 2).
2. **NFR-2 Open-source first.** No hosted SaaS dependencies on the critical path. An LLM may be a third-party model file, but it must be runnable locally on commodity hardware.
3. **NFR-3 Latency budget.** Time from webhook arrival to persisted RCA: target < 2 min warm, < 3 min cold, on the laptop runner. This budget is met (typical 25–68 s) and documented in chapter 9.

4. **NFR-4 Cost.** Total monthly run-rate target: under \$100/month, exclusive of AWS Free-Tier-eligible resources. This budget drives the decision to drop the Sprint 4 GPU migration (§11.5).
5. **NFR-5 Auditability.** Every change to the platform shall be reviewable in version control; durable design decisions shall be captured in a decisions log; daily static and live audits shall produce a written trail.

CHAPTER 4

System Architecture

4.1 Layered architecture overview

The platform is structured in three layers, illustrated in figure 4.1.

Layer 3 — AI triage FastAPI service, qwen2.5:7b on Ollama, dashboard, SMTP escalation
Layer 2 — MCP bridge prometheus-mcp, loki-mcp, jaeger-mcp, rca-history-mcp, deploy-events-mcp
Layer 1 — Observability foundation Prometheus, Loki, Jaeger, Grafana, OTel Collector, node-exporter, cAdvisor, Drain3 poller
Layer 0 — Telemetry sources Spring Boot (OTel agent), Kong (OTel plug-in), MySQL, k3s pods, host metrics

Figure 4.1. Layered architecture. Each layer consumes only the layer immediately below it; the MCP-only invariant is the rule that formalises this for the AI layer.

The arrangement is deliberately conservative. Conventional alerts flow up from layer 0 through Prometheus and Grafana in the standard way; the AI layer is an additional consumer that does not displace the existing alert routing. If the AI layer is taken down, the platform degrades gracefully to a standard Grafana-alerting setup; the inverse is not true.

4.2 Runtime topology

After the Sprint 2 migration the platform’s runtime topology distributes across three classes of host:

1. **Monitoring host** (observability-rca-monitoring on us-east-1, EC2 t3.large).
Runs the observability stack as a seven-container Docker Compose deployment: Prometheus, Loki, Jaeger, Grafana, OTel Collector, cAdvisor, node-exporter.
2. **Application host** (observability-rca-k3s on us-east-1, EC2 t3.xlarge).
Runs the application workload (Spring Boot, Kong, MySQL, front-end) on a single-node k3s cluster. A node-exporter DaemonSet provides host-level metrics from the k3s side.
3. **AI host** (the engineer’s laptop, WSL2 + Docker Desktop, NVIDIA GTX 1060 6 GB).
Runs the triage service, the five MCP servers, and Ollama with qwen2.5:7b-instruct loaded. The laptop is reachable over a Tailscale tailnet and addressed by MagicDNS (adolin-wsl).

All three hosts share a Tailscale tailnet that also contains the operator’s controller node. WireGuard handles the transport; the public IPs of the AWS instances are not exposed for service traffic.

4.3 Data flow: alert to RCA

The end-to-end flow of an alert through the platform proceeds in ten stages, summarised in figure 4.2.

Stage	What happens	Module
1	Webhook arrives from Grafana or Drain3 poller	<code>main.py</code>
2	Two-tier deduplication and suppression	<code>dedup.py</code>
3	Context gather (Prom, Loki, Jaeger, Drain3, RCA history)	<code>context.py</code>
4	Numeric metric interpretation	<code>metric_interpreter.py</code>
5	Exemplar injection (1 of 14 archetypes)	<code>exemplars/</code>
6	LLM call (structured output, qwen2.5:7b)	<code>llm_client.py</code>
6b	Response validation (surface-only, hypothesis-menu, blocklist)	<code>response_validator.py</code>
6c	Bounded-agency retry (one MCP call max)	<code>bounded_agency.py</code>
6d	Confidence clamp (strip actions if untrustworthy)	<code>pipeline.py</code>
7	Action-template fallback (if RCA actions empty)	<code>action_templates.py</code>
8	Persist to <code>rca_history</code> , dispatch email	<code>rca_store.py</code> , <code>notifier.py</code>

Figure 4.2. The ten-stage triage pipeline. Stages 6/6b/6c/6d together form the hallucination firewall.

This pipeline is described stage-by-stage in chapters 6 and 7; the validator and confidence-clamp mechanisms introduced in stages 6b and 6d are central to the project’s hallucination-firewall theme and are treated at length in chapter 7.

4.4 The MCP-only data-access invariant

MCP-only invariant

Every datum that the LLM sees in its prompt must arrive through an MCP server. Direct `httpx` or `requests` calls to Prometheus, direct `aiosqlite` reads of the RCA history database, and direct in-process `from app.exemplars import ...` imports are all forbidden in the LLM-adjacent code paths.

The invariant serves two purposes. First, it constrains provenance: every fact the LLM is shown can be traced back to a specific MCP tool call, with timestamps and response codes; there is no ambiguity about “where did the model see this”. Second, it removes a class of hallucination root causes; an in-process Python import can succeed silently and surface stale or partial data, where an MCP call has an explicit error envelope and is observable in the bridge layer’s metrics.

Three breaches of the invariant were detected during the `0b215ef3` incident on 2026-04-29 and form the scope of the Epic-4 hallucination-firewall stories (S3-HF-04 / S3-HF-05 / S3-HF-06). See chapters 8 and 7 for the diagnosis and fix plan.

CHAPTER 5

Sprint 1 — Observability Foundation

5.1 Sprint goal and topology

Sprint 1 ran 2026-03-10 to 2026-03-30. The goal was to execute on the first half of the Sprint-0 hypothesis: stand up a full-stack observability platform for the representative web stack using open-source tools and deployable by anyone with the playbook. No AI layer is present at this stage.

The Sprint-1 topology used three on-premises VMs: `monitoring-vm` (.10), `application-vm` (.11), `network-vm` (.12), on subnet 192.168.127.0/24. All three were standard Ubuntu LTS hosts with Docker installed; the entirety of the stack was deployed as Docker Compose services under Ansible control.

5.2 Ansible role layout

One role per service was established, each idempotent and parameterised, with a Docker Compose template, environment file generation, and a health-check task:

```
roles/prometheus, roles/loki, roles/jaeger, roles/grafana, roles/otel_collector, roles/kong,
roles/spring_boot, roles/cadvisor, roles/node_exporter, roles/mysql.
```

A single playbook (`playbooks/site.yml`) brought a freshly provisioned set of VMs to a fully-observable state in approximately fifteen minutes. The playbook is part of the project’s reproducibility evidence and is documented in `monitoring-docs/reproducibility-guide.html` and `build-guide.html`.

5.3 Three-pillar telemetry

- **Metrics** via Prometheus, scraping Spring Boot’s `/actuator/prometheus`, Kong’s statistics endpoint, node-exporter on each VM, and cAdvisor on the application-VM.
- **Logs** via Loki, with Promtail tailing Docker container logs on each VM and shipping them with a small set of well-controlled labels (`service_name`, `instance`, `container`).
- **Traces** via the OpenTelemetry Collector, receiving spans from the OTel Java agent injected into the Spring Boot process at start time and from the OTel plug-in installed at the Kong gateway. Trace identifiers propagated end-to-end so a single request appeared as one trace from gateway to upstream.

5.4 Grafana and initial alerting

Grafana was provisioned via Ansible templates and shipped with four canonical dashboards (service overview, alert overview, log explorer, trace explorer) installed under `/var/lib/grafana/dashboards`. Initial alerting used Alertmanager, with metric-threshold rules on CPU, memory, disk, and target-down state, routed through a Gmail SMTP contact point. Alertmanager was removed

in Sprint 2 (decision D6) in favour of Grafana unified alerting with a single `triage-webhook` contact point feeding the AI triage service.

5.5 Sprint 1 outcome

A reproducible, three-pillar observability stack for a representative web application, deployable from clean VMs in approximately fifteen minutes via a single Ansible command. No application source-code modification required; instrumentation is entirely external (OTel agent, OTel plug-in, scrape endpoints, Promtail). This is the foundation on which all subsequent sprints build.

The carry-forward items at sprint close were: the three-VM topology (superseded in Sprint 2 by AWS EC2 + k3s); Alertmanager (removed in Sprint 2 by D6); the Gmail SMTP contact point (replaced by the triage service's own SMTP send); and the on-premises IP address space (replaced by a Tailscale tailnet).

CHAPTER 6

Sprint 2 — AWS Migration and AI RCA MVP

6.1 Sprint goal

Sprint 2 ran 2026-03-31 to 2026-04-23, with a follow-on Epic 5 running to 2026-04-29. The goal was to move the platform off on-premises VMs onto AWS (Terraform-provisioned EC2), migrate the application workload to k3s, and add an AI layer that produces structured root-cause analyses from incoming alerts. The on-premises foundation from Sprint 1 remains useful as a reproducibility target; the production-shaped deployment lives on AWS.

The sprint deadline moved twice (April 9 → April 23), captured in the strikethrough trail in the project’s progress tracker. Final landing: 2026-04-23.

6.2 Epic 1 — AWS Infrastructure

A Terraform configuration in the `provisioning-monitoring-infra` repository manages the entire AWS estate: a VPC with public and private subnets, two EC2 instances (`t3.large` for monitoring, `t3.xlarge` for k3s), an RDS `db.t3.micro` MySQL instance, Elastic IPs on the durable hosts, security groups parameterised by Tailscale CIDR plus a public-UI CIDR for Grafana / Prometheus / Loki / Jaeger UI access, IAM roles per instance, an S3 bucket for Loki cold storage and a second for Terraform state. The Ubuntu AMI was pinned to a specific image identifier to remove silent drift between runs.

6.3 Epic 2 — AI RCA System MVP

This epic introduces the largest new surface in the project. It comprises:

- **Five MCP servers** (`monitoring-mcp-servers/`). Each is a small Python process that exposes three to four tools over a JSON-RPC transport with a Prometheus-style `/metrics` endpoint for observability of the bridge itself.
- **FastAPI triage service** (`monitoring-triage-service/`). Webhook receivers for Grafana and Drain3, an asynchronous ten-stage pipeline, structured-output LLM calls via Ollama, deduplication and suppression, a dashboard, and SMTP escalation.
- **Cross-pillar correlation.** The `context.py` module fans out to three MCPs in parallel before the LLM call, building one pre-gathered evidence pack the LLM consumes. This keeps the model out of the loop on which queries to run; the platform decides.
- **Drain3 log-template clustering.** Server-side over Loki, surfacing novel templates as a fourth signal alongside metrics, logs, and traces.
- **Local LLM.** `qwen2.5:7b-instruct` via Ollama, initially on a `g4dn.xlarge` EC2 (de-commissioned), then moved to the laptop’s GTX 1060 on 2026-04-23 (still the production runner today).

6.4 Epic 3 — Kubernetes migration

The application workload was moved from the application-VM Docker Compose into a single-node k3s cluster on `observability-rca-k3s`. Helm charts were authored for the Spring Boot back-end, Kong, and the front-end. The `xanmanning.k3s` Ansible Galaxy role provisions k3s as a `systemd` unit; a node-exporter DaemonSet provides host-level metrics from the k3s side.

The monitoring stack stayed on Docker Compose. Decision D11 records the rationale: the monitoring VM is operationally stable, deployable from Ansible, and gains nothing structural from migrating onto Kubernetes; the cost of the migration (Helm charts for nine services, persistent-volume claim management, Kubernetes-specific operational learning curve) exceeded the benefit.

6.5 Epic 4 — AI Triage Hardening

This epic raises the AI triage layer from “MVP that responds to webhooks” to “service operators can trust”: a two-tier deduplication layer (in-memory fingerprint plus a pre-LLM history-based suppression — decision D4), structured-output schema enforcement at the LLM call layer (D12), an error envelope on every MCP response, pipeline timeouts, the operator dashboard, the email notifier with a confidence-bordered card, and a first pass at action-template fallback.

6.6 Epic 5 — UEBA-flavoured stories (post-sprint)

Five UEBA-inspired stories ran past sprint close as a coherent in-flight workstream and were ultimately folded into Sprint 3 as the “RCA Quality v2 close-out” epic. The net effort was approximately forty-nine story points shipped after sprint close. These stories are summarised in table 6.1.

Story	What it added	Outcome
US-5.1 Phase A	Per-service Drain3 — dictionary of <code>service</code> → <code>TemplateMiner</code> with file-backed state per service.	Shipped 2026-04-28
US-5.1 Phase B	Entity baselines — 7-day rolling per-service-metric quantiles + σ -claim helper.	Shipped 2026-04-28; verification closed by S3-HF-04
US-5.3	Closed-loop feedback override/confirm + lazy <code>triage_precision</code> / <code>triage_recall</code> metrics.	Shipped 2026-04-28
US-5.4	Adaptive thresholds on three flappiest rules (same-hour-7-days-ago + 3σ baseline).	Shipped 2026-04-27
US-5.8	Recurrence gates (pre-LLM + post-LLM, Medium-CPU/Mem opt-in).	Shipped 2026-04-28

Table 6.1. Epic 5 stories carried past Sprint 2 close.

6.7 Sprint 2 outcome

The transition from “we have observability” (Sprint 1) to “the observability platform produces structured, validated RCAs end-to-end with operator-feedback closure” (Sprint 2 + Epic 5). The pipeline that runs in production today is the Sprint-2 architecture with Sprint-3’s quality and safety mechanisms layered on top. The test suite reached 178/178 at sprint close on the triage service.

CHAPTER 7

Sprint 3 — Quality, Feedback, and Hallucination Firewall

7.1 Sprint shape

Sprint 3 runs 2026-04-23 to 2026-05-19 (extended from the original 05-08 close). Four epics:

1. **Epic 1 — RCA Quality v2 close-out (49 SP)**. Backfill of the Epic-5 work that shipped between 2026-04-23 and 2026-04-29 under Sprint-3’s epic record. Twelve of thirteen stories Done; one cleanup story remains.
2. **Epic 2 — Operator feedback loop (7 SP)**. Schema extension on the existing US-5.3 feedback table; new endpoints; four new MCP tools on `rca_history_mcp`; an in-email rating form. Not started at the time of writing.
3. **Epic 3 — Drain3 baseline lifecycle (6 SP)**. Wire the dormant S3 plumbing in `drain_analyzer.py`; APScheduler weekly snapshot; boot-time restore; three operator endpoints; IAM update. Not started at the time of writing.
4. **Epic 4 — Hallucination firewall + trace depth (15 SP)**. Added on 2026-04-29 after the 0b215ef3 incident. Two themes: (a) close the three MCP-only-invariant bypasses on the LLM-gathering path; (b) wire the `get_trace(trace_id)` MCP tool that exists but is never invoked. Five SP shipped same-day as the incident; ten SP remaining.

The sprint contains a deliberate twenty-day investigation window (2026-04-30 to 2026-05-19) in the middle of its calendar; the work done during that window is described in §7.10.

7.2 The 4-axis RCA quality rubric

To make “good RCA” tractable as an engineering target, the project defines a four-axis rubric used to score every produced RCA, both in chaos-run automation and in the daily live audit:

1. **Cause-named** — the RCA names a root cause, not just a restatement of the alert (e.g. “connection pool exhaustion on `spring-boot`” rather than “the alert indicates high latency”).
2. **Evidence-cited** — the cause is supported by specific metric values, log lines, or trace attributes drawn from the pre-gathered context.
3. **Actionable** — the RCA carries either operationally valid suggested actions or, in the suppressed case, valid read-only diagnostic steps.
4. **Prose-shape** — the RCA leads with the cause rather than with a PromQL expression, observation, or “based on context” opener (the philosophy stated by decision D19).

Each axis is scored on a 0–1 scale; the unweighted mean is the overall RCA quality. The rubric is implemented in `monitoring-project/scripts/chaos/lib/rca_scorer.py` and used both by the chaos-injection harness and the daily live audit.

7.3 The exemplar library (D17)

Decision D17 — Exemplar injection

The LLM is shown one of fourteen canonical RCA archetypes as a structural target before it sees the evidence pack. Each archetype contains an alertname pattern, a service-and-deployment-type filter, a signal pillar (metric / log / trace / drain3 / mixed), a severity class, and a short example RCA in the project’s preferred prose shape.

The fourteen archetypes are: oom-loop, upstream-latency-attribution, service-self-p95-saturation, critical-resource-saturation, otel-collector-degradation, synthetic-blip-dismiss, cascade-incident-disk-loki, drain3-novelty-post-deploy, bounded-agency-resolves-inconclusive, adaptive-threshold-noop, closed-loop-feedback-override, crashloop-bad-config, network-firewall-tls-cert-expiry-pre-failure. A default generic-sre-shape fallback is used when none match.

Selection is via `exemplars.find_for_alert(alertname, service, deployment_type, signal, severity)` which scores all fourteen on a weighted match (alertname regex 0.10, services 0.40, deployment-type 0.30, signal 0.20, severity 0.10) and returns the highest. The matched exemplar is rendered as a **## Reference exemplar** section *before* the **## Pre-Gathered Context** in the prompt — the placement is load-bearing and was confirmed during the investigation window (§7.10) to outperform post-context placement on the 4-axis rubric.

7.4 Adaptive thresholds (D18)

Static numeric thresholds (`p95 > 1000ms`, `cpu > 80%`) generate floods of false positives during off-hours and miss real anomalies during peak traffic. Real traffic isn’t stationary; rules shouldn’t pretend it is. Three Grafana rules (`HighP95Latency`, `HighKongP95Latency`, `MediumCpuUsage`) were converted to a same-hour-7-days-ago baseline plus a 3σ envelope. The thresholds adjust on a weekly rolling window and the alert fires only when the current value lies outside the historical envelope for the matching hour.

7.5 RCA prose philosophy (D19)

Decision D19 captures the position that *telemetry is the means, not the end*: an RCA paragraph that leads with a PromQL expression or an observed metric value is reading like the alert query, not like an investigation. Three reinforcing surfaces were updated to enforce a cause-first prose shape:

- **SYSTEM_PROMPT** rule A and a new rule J explicitly instruct the model to lead with the cause; the previous version of rule A had instructed the model to lead with the PromQL expression and observed value, an unintended training surface for surface-only output.
- All eleven exemplar `rca` paragraphs were rewritten to match the cause-first shape.
- A new validator rule scans the produced RCA against a set of `_SURFACE_ONLY_LEDE_PATTERNS` regexes. Matches trigger a retry under the bounded-agency path.

7.6 Closed-loop feedback (D20)

Operator overrides on RCAs were previously silent: the LLM produced a verdict; the operator either agreed or didn’t; the system never learned. Without that signal, every prompt, exemplar

or threshold change was unmeasurable.

Decision D20 introduces a closed-loop layer. A `feedback` table records the operator’s actual verdict ($\pm$ a free-text `actual_root_cause`). Two endpoints expose the surface (`/feedback/override` and `/feedback/confirm`). Two Prometheus metrics, `triage_precision` and `triage_recall`, are computed lazily from the feedback table. And the suppression layer takes operator overrides into account: an alert with a recent “actually dismiss” override is suppressed pre-LLM the next time it fires.

7.7 Recurrence gates (D21)

An alert that fires every ninety minutes is too slow for fingerprint deduplication (10-minute window) but too noisy to page someone every time. Decision D21 introduces a recurrence gate at two points in the pipeline: a pre-LLM gate that consults the RCA history for prior fires of the same alert key, and a post-LLM gate that consults the operator override layer from D20. Critical-severity alerts bypass the gate. Per-rule opt-in is via a Grafana rule annotation `recurrence_gate`; `MediumCpuUsage` and `MediumMemoryUsage` are currently opted in.

7.8 Per-service Drain3 template trees (D23)

The original Drain3 integration used a single global `TemplateMiner`. Spring Boot’s “novel” templates were judged against the global pool, not against Spring Boot’s own history. A line that’s commonplace for `spring-boot` but unusual for `kong` looked the same as a line that’s unusual for both. Decision D23 introduces a per-service split: a dictionary `dict[service → TemplateMiner]` with `FilePersistence` per service, and a migration path that converts the pre-split `drain3_state.bin` into a service-keyed family.

7.9 The hallucination firewall epic (Epic 4)

Epic 4 was added on 2026-04-29 after the `0b215ef3` incident (chapter 8). The case-study chapter covers the incident itself; this section summarises the resulting stories:

- **S3-HF-01 (Tier 0, 1 SP, shipped 2026-04-29).** Confidence clamp strips `suggested_actions` when the clamp fires and emits a read-only `diagnostic_steps` field instead. Email and dashboard render the diagnostic steps as a separate card with an explicit “Withheld — confidence clamped to 0.40” notice.
- **S3-HF-02 (1 SP, shipped 2026-04-29).** Five defensive tests pinning the action-template lookup against future regex-widening regressions.
- **S3-HF-03 (Tier 2, 2 SP, shipped 2026-04-29).** Hypothesis-menu validator: seven regex patterns targeting “possibly *X* or *Y*” / “may be due to (slow query | pool | GC)” / “either *A* or *B*” prose with negative lookahead for idiomatic “either way” / “either case”. Companion cause-evidence rule that tokenises both RCA and evidence with `[a-z]{4,}` (so `hikaricp_connections_active` splits into `[hikaricp, connections, active]` for cross-matching) and requires non-stop-word overlap.
- **S3-HF-04 (1 SP, in flight).** Route `entity_baselines.py:144` through `prometheus-mcp` instead of direct `httpx` to Prometheus `/api/v1/query`. Closes one of three MCP-only-invariant bypasses.

- **S3-HF-05 (1 SP, blocked on S3-HF-06).** Route the `bounded_agency.py` RCA-history paths through `rca-history-mcp` instead of direct `aiosqlite`.
- **S3-HF-06 (2 SP).** Extend `rca_history_mcp` with quality-rated tools: `get_similar_decisions(min_qu` and `get_low_rated_examples_for_alert`. Combined with S3-FB-01 in one PR.
- **S3-HF-07 (Tier 1, 3 SP).** Deep trace gather via `get_trace(trace_id)` MCP. The pipeline currently gathers Jaeger data at the service-boundary level only and can identify “upstream is slow” but cannot name what is slow. With trace depth, the LLM sees per-span attributes (`db.statement`, `http.target`, error tags) for the failing trace. Pre-step (executed during the investigation window): sample 12 live Spring Boot traces to confirm JDBC parameterisation. Result: `db.statement` spans use bind parameters (`?, ?, ?`), so PII redaction is not a blocker.
- **S3-HF-08 (2 SP, blocked on S3-HF-04 and S3-HF-05).** MCP-integrity continuous-integration lint forbidding `httpx/requests/direct-DB` calls outside `_mcp_call` helpers and the MCP servers themselves.
- **S3-HF-09 (Tier 4, 2 SP).** Chaos scorer 5th axis, labelled corpus seed (the `0b215ef3` incident itself), and a CI regression gate. Must land before Sprint 4’s Tier 3 iterative agentic gather, which depends on the Tier 4 measurement scoreboard.

7.10 The 20-day investigation window (2026-04-30 to 2026-05-19)

After Epic 4 was added on 2026-04-29 and the Tier-0/2 stories shipped the same day, the team deliberately slowed down before the larger MCP refactors (S3-HF-04 through S3-HF-09). Approximately three weeks went into prototyping output-quality improvements in scratch branches and refreshing documentation, with no commits to `main` by design — the goal was to know which ideas were worth the refactor cost before paying it. Nothing structural shipped during the window; the production triage container stayed on commit `d6d10ef` from 2026-04-29, and the seventeen daily static audits all reported the test suite green at 226/226 with the same set of seven minor doc-drift items. Eight candidate output-quality improvements were A/B-tested against a twenty-eight-decision sample drawn from the live-audit log and chaos runs 3 and 4; the outcomes are summarised in table 7.1.

7.11 Documentation refresh

A substantial documentation refresh landed on 2026-05-19 as the window closed:

- A new canonical `sprint-history.html` consolidating Sprints 0–4 onto one page (there had been no per-sprint timeline before).
- A new `solution-brief.html` as a report-facing one-pager, built around the `0b215ef3` case study, the validator details, the fourteen-archetype table, and a glossary.
- Metric-name doc-vs-code mismatches reconciled in `trriage-service.html` §7.
- Consistency passes across `architecture.html` and `system-explained.html`.
- Tested-and-rejected experiments folded back as footnotes to D17 / D19 / D21 in `decisions-log.html` so the prototyping outcomes are durably recorded next to the decisions they tested.

Idea	Result	Decision
Exemplar / context section ordering — exemplar after context.	Cause-named score dropped 0.71 $\rightarrow$ 0.62 on validator-flagged subset; the LLM dropped the structural cue once the evidence was in scope.	Keep pre-context order
Evidence-pillar weighted re-ranker.	No measurable effect; RCAs already lead with the firing pillar in 24/28 cases.	Shelved
BM25 retrieval prototype.	Degenerate over fourteen archetypes; top-1 hit always matched <code>exemplars.find_for_alert</code> . Sprint-4 candidate #3 stays as scoped.	Confirms retriever choice is not the gate
Hypothesis-menu validator threshold sweep.	0/120 false positives at strict mode; warn-only adds no catches.	Default strict stays
Confidence-clamp threshold (0.30 / 0.40 / 0.50).	Below 0.40 dampens prose; above 0.40 actions leak through.	0.40 retained
Trace-depth feasibility scan on 12 live Spring Boot spans.	<code>db.statement</code> uses bind parameters not literal values.	S3-HF-07 has no PII blocker
RCA prose A/B (cause-first vs. observation-first).	Observation-first regresses on cause-named when evidence is thin.	Cause-first retained (reinforces D19)
Pipeline-duration per-stage instrumentation.	Drafted in scratch branch.	Did not merge; depends on S3-HF-05

Table 7.1. Investigation-window outcomes (2026-04-30 to 2026-05-19). Each candidate was scored on the 4-axis RCA quality rubric over a twenty-eight-decision sample.

7.12 The bounded-agency retry

The bounded-agency retry is the platform’s mechanism for handling genuinely data-starved RCAs: if the validator flags the first pass, the LLM is given exactly one additional MCP call from a six-tool whitelist (Prometheus instant query, Loki range, Jaeger get-traces, RCA-history lookup, deploy-events, and `get_trace`). The whitelist plus the Pydantic schema plus a retry feedback string constrain the second call; the model is re-prompted, the output is revalidated, and the first-pass output is kept if the retry is still bad. This is decision D15, “one MCP call, not a chain” — a deliberately constrained version of agentic gather chosen for predictability and bounded latency, with the unbounded *iterative agentic gather* scoped to Sprint 4 once the Tier-4 measurement scoreboard is in place.

CHAPTER 8

Case Study: the 0b215ef3 Incident

8.1 The alert

On 2026-04-29, a `HighKongP95Latency` alert fired for the `spring-boot` upstream. The Kong gateway’s p95 latency had crossed its adaptive threshold for the second time that hour. The webhook arrived at the triage service on the laptop at `adolin-wsl` and produced the RCA with identifier `0b215ef3`.

8.2 The bad RCA

The produced RCA carried:

- **Verdict.** `escalate`.
- **Confidence.** `0.55` (moderate).
- **Root-cause prose.** A hedged “hypothesis menu”: “This could be a slow query in the `products` service, a connection-pool exhaustion event, or a garbage-collection pause; the latency profile is consistent with any of the three.”
- **Suggested actions.** Three templated `kubectl` commands operating on a `products` deployment that does not exist (the system runs Spring Boot, not a microservice called `products`).
- **Evidence.** Two Prometheus queries against `kong_http_requests_total` with their text rendered but no values cited; one trace identifier without `db.statement` or per-span attributes.

The hypothesis menu was the visible symptom. The deeper failures were structural: the LLM had been given the latitude to invent a `products` service that does not exist; the confidence value was not low enough to trip the existing clamp at `0.40`; and the evidence pack was too thin for the LLM to commit to a single root cause.

8.3 Forensic analysis: three MCP-invariant violations

The post-incident analysis identified three distinct code paths that violated the MCP-only invariant and contributed to the bad output:

1. `monitoring-triage-service/app/entity_baselines.py:144` was using a direct `httpx` call to Prometheus’s `/api/v1/query` endpoint, bypassing `prometheus-mcp`. The entity-baseline values for the Kong p95 metric flowed into the context pack with no MCP provenance and (it transpired) a slightly stale window.
2. `monitoring-triage-service/app/bounded_agency.py` was using direct `aiosqlite` reads of the `rca_history` table for `retry-tool` dispatch, bypassing `rca-history-mcp`. This meant

the retry path could not benefit from the `get_similar_decisions(min_quality)` filter that would have surfaced operator-confirmed similar cases.

3. `monitoring-triage-service/app/llm_client.py` and the exemplar consumers used `in-process` from `app.exemplars` `import ... imports` rather than an MCP tool. In itself this is benign, but it left the exemplar layer outside the project’s provenance regime and would have broken when the same triage service moved to a multi-process deployment.

A fourth, related, finding was that the existing `get_trace(trace_id)` MCP tool — written months earlier — had never been called from the pipeline. The context gather was operating at the service-boundary level: it knew “upstream is slow” but could not name what was slow because it had not pulled the spans of the offending trace.

8.4 The tiered response

A planning agent produced a five-tier response plan, captured in the session handoff and translated into the Epic 4 stories listed in §7. In summary:

- **Tier 0** — strip suggested actions when the confidence clamp fires; emit read-only diagnostic steps instead. Shipped same day as S3-HF-01.
- **Tier 1** — deep trace gather via `get_trace(trace_id)`. Scoped as S3-HF-07; in flight.
- **Tier 2** — hypothesis-menu validator and cause-evidence rule. Shipped same day as S3-HF-03.
- **Tier 3** — iterative agentic gather with hypothesis-tree state. Deferred to Sprint 4; depends on Tier 4 being in place first.
- **Tier 4** — chaos scorer 5th axis, corpus seed, regression gate. Scoped as S3-HF-09.

The same-day work shipped as commits `b0f1d0c` and `d6d10ef` on the triage service, plus accompanying commits on the monitoring-docs and Jira-CSV repositories. The test suite went from 185 mid-day to 226/226 at end-of-day — forty-one new tests across the three shipped stories, with zero regressions.

8.5 Lessons captured

The incident is the canonical case study in the project for two reasons. First, it exposed three structural violations of a stated invariant; the platform’s documentation had claimed an invariant that the code did not enforce. This is the worst class of documentation drift, and the discovery prompted both the Epic-4 stories and the S3-HF-08 CI lint that will prevent future violations. Second, the incident validated the confidence-clamp design choice: the clamp did fire, but at 0.55 the configured ceiling of 0.40 was high enough that the bad actions leaked through. The clamp behaviour was strengthened (strip actions, emit diagnostic steps) rather than re-thresholded; the investigation-window threshold sweep (table 7.1) confirmed 0.40 was the right ceiling.

CHAPTER 9

Evaluation

9.1 Test-coverage progression

The triage-service test suite size over the project’s lifetime is a useful coarse measure of the engineering load behind the visible features. Table 9.1 summarises the milestones.

Date	Suite size	Milestone
2026-04-27 evening	89	Exemplar library (D17) tests in place
2026-04-28 mid-day	95	RCA-prose validator (D19) tests added
2026-04-28 EOD	178	Full RCA quality chain (F-1 to F-5) live
2026-04-29 EOD	226	S3-HF-01/02/03 shipped
2026-05-19	226	Held green across 17 daily audits

Table 9.1. Triage-service test-suite progression.

The point at which the suite size more than doubled in a single 24-hour window ($95 \rightarrow 178$) marks the F-1 to F-4 RCA-quality work described in chapter 6. The +48 over the 2026-04-29 day ($178 \rightarrow 226$) is the Epic-4 same-day stories.

9.2 RCA quality on the 28-decision sample

The investigation-window evaluation produced a baseline RCA quality score on a 28-decision sample drawn from the live-audit log and chaos runs 3 and 4. On the 4-axis rubric:

Axis	Mean score
Cause-named	0.78
Evidence-cited	0.84
Actionable	0.71
Prose-shape	0.89
Overall (unweighted)	0.81

Table 9.2. Baseline RCA quality on the 28-decision sample (higher is better). Actionable is the lowest score because of the deliberate strip-actions-on-clamp behaviour (S3-HF-01), which lowers the actionability score but raises operational trust.

9.3 Chaos-run outcomes

The chaos-test harness in `monitoring-project/scripts/chaos/` runs five scenarios (`target_down`, `high_memory_usage`, `high_cpu_usage`, `drain3_anomaly`, and the new `0b215ef3`-style corpus seed). Each run induces a controlled failure, polls `/decisions` for a matching alert, scores the RCA, and regenerates `chaos-report-YYYY-MM-DD.html`.

Run 3 (2026-04-28) captured 0/4 because pod-scoped chaos did not move host-level metrics. This finding produced the pod-level alert rules (`PodHighMemoryUsage`, `PodHighCpuUsage`) added the same evening. Run 4 (2026-04-28 PM) captured 0/2, this time for known reasons: container

restart churn split timeseries across `cgroup` identifiers; cumulative CPU expressions did not match their denominator labels. Both are tractable thirty-minute fixes filed as Sprint 4 candidate #5.

9.4 Audit-corpus consistency

The daily static audit ran 17 times during the investigation window (2026-04-30 to 2026-05-19), all green at 226/226 with the same seven-item doc-drift list carried forward day-on-day — no new drift accumulated. The live audit (cron on `claude-controller`) last ran on 2026-04-29; the cron drift is filed as resilience item R-1 (chapter 11).

9.5 Latency budget

The Sprint 2 verdict-story field note recorded an end-to-end pipeline latency of 67 995 ms (≈ 68 s) on the laptop runner with `qwen2.5:7b-instruct`, against a historical k3s-on-CPU baseline of 20–40 *minutes* per alert. Sustained steady-state latencies on the laptop are 25–35 s warm. Five recent fires were anomalously slow (323, 399, 533, 755, 1081 s) and the per-stage instrumentation drafted during the investigation window will surface whether the cause is bounded-agency retries, LLM prefill saturation, or MCP tail latency; the instrumentation is held until S3-HF-05 ships so the retry path is properly attributable.

9.6 Operational steady state

The qualitative observation that the platform held a steady state across a twenty-day low-traffic window is itself an evaluation result: no incidents observed; AWS instances stayed up; `ai-triage-service` on the laptop required exactly one manual restart over the period (caused by Docker restart-backoff after a WSL sleep, not by a platform fault). This validates the architectural choice of running the AI layer separately from the production infrastructure: a transient laptop fault does not affect the observability stack, only the AI-assisted RCAs.

CHAPTER 10

Design Decisions and Rationale

The project maintains a durable decisions log (`monitoring-docs/decisions-log.html`) recording the twenty-three substantive design or rule-of-thumb decisions taken during construction. Each entry follows a stable structure: the problem observed, the constraints, the decision, and the verification that closed it. Table 10.1 summarises the catalogue.

Table 10.1. Catalogue of durable design decisions D1–D23.

ID	Title	One-line decision
D1	LLM hedging with “insufficient data”	Treat “insufficient data” as a verdict, not a substitute for one; persist it but never escalate on it.
D2	Empty <code>suggested_actions</code> on most RCAs	Add the action-template fallback layer; never emit an RCA with both empty actions and <code>rca_quality=actionable</code> .
D3	Observed value picked from the wrong <code>refId</code>	Default to <code>refId=A</code> ; require an explicit override in the alert rule to use a different reduced expression.
D4	<code>llm_confidence</code> never persisted	Persist confidence; do not surface a row without it; use the value to gate the action-template fallback.
D5	<code>rca_quality</code> lied when actions+evidence empty	Compute quality from evidence and action shape, not from the verdict string.
D6	Overnight email incident — rogue k3s AI stack	Decommission k3s-hosted AI; move AI onto laptop; remove Alertmanager; unify alerting in Grafana.
D7	Deployment-type mismatch	Carry an explicit <code>service_deployment_type</code> map and template actions by deployment type.
D8	Drain3 anomalies never fired into triage	Wire a synthetic webhook from the Drain3 poller into <code>/webhook/drain3</code> ; relax thresholds (rate 0.10 → 0.02, lines 100 → 20).
D9	Flapping alerts consumed $N \times$ LLM calls	Two-tier dedup; pre-LLM fingerprint suppression; recurrence gates (later, D21).
D10	Correlated alerts rendered but not prompted	Include the correlated-alerts list in the prompt’s pre-gathered context.
D11	Monitoring stack stays on Docker Compose	Migrate the application workload to k3s; keep observability on Docker Compose.
D12	Structured outputs vs. JSON mode	Use structured-output schema enforcement at the LLM layer; retry on schema parse failure.
D13	Pre-LLM metric interpreter	Move numeric reasoning out of the LLM into a deterministic Python module; pass interpreted values into the prompt.
D14	Downtime backfill from Grafana annotations	At startup, query Grafana annotations for the missing window, replay them through the pipeline (bounded by a max gap).
D15	Bounded-agency retry	Exactly one MCP call from a six-tool whitelist; revalidate; keep first-pass if retry still bad.

(continued)

ID	Title	One-line decision
D16	UEBA framing for Sprint 2 Epic 5	Add the entity-behavior layer (per-service Drain3, baselines, feedback) as an extension to the event-centric core.
D17	Exemplar library	Inject one of fourteen canonical RCA archetypes as structural scaffolding before the evidence pack.
D18	Adaptive thresholds	Replace static numeric thresholds on the three flappiest rules with same-hour-7-days-ago + 3σ baselines.
D19	RCA prose quality	Telemetry is the means, not the end — RCAs must name a cause, not restate the alert.
D20	Closed-loop feedback	Capture operator overrides; compute precision/recall; consume overrides in the suppression layer.
D21	Recurrence gates	Pre-LLM + post-LLM hysteresis for opted-in alerts; critical severity bypasses.
D22	SESSION_HANDOFF kept controller-local	Untrack from public repo; scrub history; keep as a local scratchpad.
D23	Per-service Drain3 template trees	Dictionary of <code>service</code> → <code>TemplateMiner</code> ; file persistence per service; migration of pre-split state.

10.1 Three decisions in depth

10.1.1 D17 — Why exemplars work where retrieval did not

The naive approach to “teach the LLM what good looks like” would have been retrieval-augmented generation over the `rca_history.db` table. D17 rejects this for two reasons. First, the historical record *contains the failures we are trying to fix* — training the model on the empirical RCA history would reinforce the surface-only and hypothesis-menu patterns the project is trying to eliminate. Second, the historical record is unlabelled at the time of the decision; without operator-feedback quality ratings, retrieval would pick by surface similarity, not by quality.

The exemplar library is a curated corpus, not an empirical one. Each of the fourteen archetypes is hand-authored by the engineering team to model the prose shape and reasoning structure the platform wants to reproduce. This is a deliberate inversion of the usual “train on what we have” approach in favour of “train on what we want”. The BM25 retrieval dry-run during the investigation window (§7.10) confirmed empirically that retrieval over fourteen archetypes is degenerate: the top-1 hit always matches the existing `find_for_alert` selector, so adding retrieval on top contributes nothing. Once the operator-feedback table accumulates quality-rated entries (Epic 2 work), curated retrieval over a quality-filtered subset becomes worthwhile — this is the Sprint 4 US-3.3 / US-3.4 scope.

10.1.2 The MCP-only invariant as a project-wide rule

The MCP-only invariant is not a technical mechanism but a project-wide rule. Its enforcement is partly social (code review, the audit reports), partly architectural (the MCP servers exist; they have error envelopes and metrics), and ultimately mechanical (the S3-HF-08 continuous-integration lint forbids the bypass patterns). The `0b215ef3` incident demonstrated the cost of relying on the social layer alone: three of the project’s documents (`prepare.html`, `system-explained.html`, `triage-service.html`) claimed an invariant the code did not enforce, and the resulting RCA carried hallucinated content because of it.

The mechanism that makes the rule durable is the combination of (a) error-envelope-shaped MCP responses, so a failed query is observable rather than silent; (b) MCP-bridge metrics, so call rates and latencies are visible in Grafana; and (c) the CI lint, so the rule has teeth on new code as well as existing. The order of operations matters: the lint cannot fire until S3-HF-04 and S3-HF-05 have closed the existing offenders, because a lint that fails on existing code is one the team will work around rather than fix.

10.1.3 Bounded agency over full agentic gather

Decision D15 introduces the bounded-agency retry: the LLM is given exactly one extra MCP call after a validator violation, from a six-tool whitelist, with a retry-feedback string and the same Pydantic schema as the first pass. This is a deliberately constrained form of agentic gather: an unbounded loop (Sprint-4 candidate #2, Tier 3) gives better RCA quality on data-starved cases but introduces unbounded latency and unbounded tool-call cost. Bounded agency is the project's currently-chosen trade-off: predictable latency, bounded LLM cost per alert, and a measurable quality improvement on validator-flagged cases.

The natural extension is Tier 3 (iterative agentic gather with hypothesis-tree state); the project scopes it to Sprint 4 explicitly gated on Tier 4 (the chaos-scorer regression gate from S3-HF-09). Shipping Tier 3 without Tier 4 in place would be a high-risk change without a measurement scaffold; this gating is a deliberate methodological choice.

CHAPTER 11

Future Work and Sprint 4 Roadmap

11.1 Sprint 4 candidate ranking

The ranked list of Sprint 4 candidates, with the dropped GPU migration removed, is reproduced in table 11.1.

Table 11.1. Sprint 4 candidates as of 2026-05-19.

#	Item	Effort	Gates
1	US-5.2 Incident correlator — 4-alert kill-chain RCA collapsed into one coherent email	1–2 d	None
2	Tier 3 — Iterative agentic gather	3–5 d	S3-HF-09
3	Curated RAG retrieval (BM25 over feedback-curated YAML)	—	Epic 2 feedback table populated
4	Weekly curation job (APScheduler, Sundays 02:00 UTC)	—	Feedback volume
5	Pod-level alert rule fixes (memory cgroup-churn + CPU label mismatch)	~30 min each	None
6	Sentence-transformers retriever fallback	—	BM25 underperforms in A/B
7	O-9 webhook auth + O-10 nightly RCA-history S3 backup	~30 min each	None
8	MCP native tool-calling rewrite	5 SP	— (no longer gated on GPU)
9	Trace-ID linkage from log anomalies	2 SP	None
10	RCA playbook templates	—	Complements Epic 2
11	US-5.7 labelled corpus + F1 tracking	—	S3-HF-09
12	L2/L3 chaos scenarios (O-12)	—	None
13	Drain3 self-monitoring loop allowlist	~30 min	None
14	Pipeline duration per-stage instrumentation	1–2 h	S3-HF-05
15	Doc debt — CLAUDE.md refresh, stale PNGs replaced	1–2 h	None

11.2 Tier 3 — iterative agentic gather

The Tier-3 design replaces the single-shot bounded-agency retry with a three-iteration tool-call loop, each iteration producing a partial hypothesis tree that the model expands on the next iteration. The hypothesis tree is stored in the prompt as a structured section; at each iteration the model is asked to expand the most promising leaf, gather supporting evidence via a single MCP call, and update the tree. Termination conditions: a confidence threshold reached, a maximum-depth limit hit, or a total latency budget exhausted.

This is materially more permissive than the bounded-agency retry; the gating on S3-HF-09 reflects the project’s view that shipping Tier 3 without an automated quality-regression gate would be imprudent.

11.3 Curated RAG retrieval (US-3.3 / US-3.4)

Once the Epic-2 operator feedback table has accumulated rated entries, curated retrieval becomes worth the engineering effort. The design: a weekly job reads the feedback table for the last seven days; entries above a quality threshold are written as YAML to a `library_curated.yaml` file; the triage prompt's exemplar section is supplemented by a small number ($k = 3$ positives, $k = 2$ anti-examples) retrieved by BM25 over that curated set. The investigation-window dry-run established that BM25 over the fourteen-archetype *base* library is degenerate but did not evaluate BM25 over an enriched library; that is the actual production scenario and will be measured once Epic 2 ships.

11.4 Resilience tier (R-1 to R-5)

The 20-day investigation window surfaced five operational-resilience items that were not blocking but are worth Sprint 4 inclusion:

- **R-1** Live-cron heartbeat metric on a Prometheus push-gateway with a `LiveAuditMissing` Grafana rule.
- **R-2** Move the live-audit cron off the ephemeral `claude-controller` onto the durable `observability-rca-monitoring` EC2 via a `systemd` timer.
- **R-3** CI lint that fails if any `sprint*-plan.md` window end-date is in the past.
- **R-4** `the-bigger-picture.md` staleness badge in the repository README, red at fourteen days.
- **R-5** `SESSION_HANDOFF.md` S3 backup, folded into the O-10 nightly-backup work.

11.5 Dropped scope: cloud GPU migration

The cloud-GPU migration to `us-west-2` (g5.xlarge A10G 24 GB, host for `qwen2.5:14b` or `32b-q4`) sat as a Sprint-4 candidate from 2026-04-27, when the G+VT on-demand vCPU quota was approved. It is **dropped on cost grounds on 2026-05-19**.

A `g5.xlarge` runs approximately one US dollar per hour on-demand, or roughly \$720 per month if left running, with aggressive auto-stop bringing the realistic monthly cost to \$200–300. This is an order of magnitude more than the laptop GTX 1060 + `qwen2.5:7b` setup that has been the production runner for twenty-five days with no quality regression surfaced in the daily audits or the investigation-window RCA scoring. The `qwen2.5:14b` / `32b-q4` upgrade path is parked indefinitely; the gating condition for revisiting the decision is either a material degradation of 7b output quality or the availability of a free or heavily subsidised GPU.

The decision has two knock-on effects. First, the MCP native tool-calling rewrite (Sprint 4 candidate #8) was previously gated on the GPU migration; the larger model is more reliable on function-calling benchmarks than the 7B and the rewrite was held until the migration shipped. With the migration dropped, the rewrite proceeds on the 7B model directly; the structured-output schema enforcement and the bounded-agency retry combine to keep tool-calling reliability operationally adequate at this scale. Second, the `us-west-2` G+VT quota approval (4 vCPU, granted 2026-04-27) remains on the account but stays unused.

11.6 Beyond Sprint 4

Beyond the ranked candidates, three longer-arc directions are worth recording for the supervisor's consideration:

- **A multi-tenant deployment model.** The current platform is single-tenant by construction. A multi-tenant deployment (e.g. one triage service per business unit at Tanger Med, with shared MCP bridges and a shared observability stack) would require namespace isolation in the RCA history, per-tenant exemplar libraries, and per-tenant confidence calibration.
- **Direct integration with Tanger Med production systems.** The platform is currently exercised against a representative web stack. Onboarding actual production services would require careful PII handling (operator feedback content, log lines), an SLO programme to set meaningful confidence thresholds, and a formal change-control process for prompt and exemplar updates.
- **Federated model evaluation.** As the project accumulates labelled RCA outcomes via the feedback layer, comparing alternative LLM choices on a held-out set becomes feasible. Models worth evaluating: `qwen2.5:14b-instruct`, `deepseek-r1:14b-distill`, `mistral-nemo:12b`. The cost-vs-quality curve will be the deciding factor; this is the natural successor to the dropped GPU migration.

CHAPTER 12

Conclusion

This dissertation has reported on the design and implementation of the CIRES Observability Platform, an AI-augmented observability solution built over three completed sprints at CIRES Technologies. The platform addresses the operational gap identified during Sprint 0 field research — that the team was finding out about production problems after the fact rather than proactively — by combining a conventional open-source observability stack with an AI triage layer that turns each alert into a structured, pre-investigated RCA.

The principal contributions of the work are:

1. **An end-to-end reference implementation.** A Terraform-provisioned, Ansible-configured, k3s-and-Docker observability stack on AWS, with a FastAPI-based AI triage service producing structured RCAs from Grafana webhooks via a locally-hosted LLM. Reproducible from clean infrastructure and operable from a single playbook command.
2. **The MCP-only data-access invariant.** A project-wide rule, partly architectural and partly mechanical (via the Sprint-4 lint), that every datum the LLM sees must arrive through an MCP bridge. The rule produces a provenance regime over the LLM’s input and is enforced via the bridge layer’s metrics, the audit reports, and a continuous-integration lint.
3. **The hallucination firewall.** A layered combination of mechanisms (exemplar injection, alert-keyed blocklist, surface-only validator, hypothesis-menu validator, cause-evidence rule, confidence clamp with action stripping, bounded-agency retry) designed to keep LLM output operationally trustworthy. Each mechanism addresses a specific failure mode that was observed empirically during construction and is recorded in the decisions log with the evidence that drove it.
4. **The closed-loop operator-feedback layer.** A schema and surface that captures operator overrides on RCAs and propagates them into the suppression layer, the precision/recall metrics, and (in Sprint 4) the curated retrieval corpus.
5. **A documented decisions log.** Twenty-three durable design decisions (D1–D23) recorded contemporaneously with the engineering work, including the problem observed, the constraints, the decision, and the verification. The log is a primary deliverable of the project and is exercisable in future audits.

The platform is in active service against a representative web stack on AWS, with daily static and live audits producing a documented trail of operational health. The remaining Sprint 3 work and the ranked Sprint 4 candidates carry the platform forward; the methodological choice to gate Sprint 4’s iterative agentic gather (Tier 3) on a measurement scaffold (Tier 4) is, in the author’s view, the most important durable lesson from the project — the cost of shipping an unmeasurable AI mechanism into production materially exceeds the cost of building the measurement first.

CHAPTER A

Repository inventory

Repository	Purpose
monitoring-project	Ansible roles, playbooks, chaos harness
provisioning-monitoring-infra	Terraform AWS state
monitoring-docs	This document, decisions log, architecture page, sprint history
monitoring-triage-service	FastAPI triage service, pipeline, validators
monitoring-mcp-servers	Five MCP bridges
monitoring-audit-results	Private — daily static + live audits
jira	Jira CSV exports and import scripts

Table A.1. Project repository layout (seven repositories).

CHAPTER B

Glossary

AIOps An informal label for the application of artificial- intelligence techniques to IT operations problems — anomaly detection, root-cause analysis, alert summarisation.

Bounded-agency retry The platform’s one-MCP-call retry layer. The LLM is given exactly one additional tool call after a validator violation; the schema and the retry-feedback string constrain the call.

Cause-evidence rule A validator rule that tokenises the RCA prose and the cited evidence with `[a-z]{4,}` and requires a non-stop-word overlap. Rejects RCAs whose cause is not supported by the cited evidence.

Confidence clamp The pipeline step that forces `confidence` ≤ 0.4 if the surface-only or hypothesis-menu validators fired, if the RCA quality is `data_starved`, or if the actions look templated. Strips `suggested_actions` when fired and emits read-only `diagnostic_steps` instead.

Drain3 A log-template clustering algorithm (a tunable variant of Drain). Used in the platform as a fourth telemetry pillar surfacing novel templates.

Exemplar archetype One of fourteen hand-authored RCA examples injected into the prompt as structural scaffolding before the evidence pack.

Grafana unified alerting The successor to Alertmanager in the Grafana stack. All alert rules and contact points live inside Grafana itself.

Hallucination An LLM output statement that is not entailed by the input context (e.g. a service that does not exist; a metric value that was not in the evidence pack).

Hypothesis menu An RCA that hedges with multiple possible causes. Detected by the S3-HF-03 validator.

MCP Model-Context-Protocol — an open specification for connecting LLM applications to external data sources and tools.

MCP-only invariant The project-wide rule that every datum the LLM sees must arrive through an MCP server.

node-exporter The standard Prometheus exporter for Unix host metrics (CPU, memory, disk, filesystem, network).

OTel OpenTelemetry, the de-facto open standard for telemetry instrumentation.

RCA Root-Cause Analysis — the structured explanation of why an incident occurred.

SRE Site Reliability Engineering — the operational discipline formalised by Google’s homonymous book and workbook.

Surface-only LEDE An RCA opening that restates the alert (e.g. leading with the PromQL expression or the observed value) instead of naming a cause. Detected by the `_SURFACE_ONLY_LEDE_PATTERNS` regex set.

Tailscale tailnet The WireGuard-based mesh that connects all hosts in the platform. MagicDNS provides stable host names.

Triage In the platform's vocabulary, the AI layer that consumes alerts and produces RCAs (as opposed to the human-facing triage practice in incident response).

CHAPTER C

Sprint chronology summary

Sprint	Window	Headline outcome
0	Feb 2026 – early Mar 2026	Problem statement crystallised; solution hypothesis (observability + AI triage) chosen.
1	2026-03-10 – 2026-03-30	On-prem three-VM observability stack deployable in 15 min via Ansible.
2	2026-03-31 – 2026-04-23	AWS migration, k3s for the application workload, AI RCA MVP (5 MCP servers + FastAPI triage + Ollama).
2-ext	2026-04-23 – 2026-04-29	Epic 5: per-service Drain3, entity baselines, closed-loop feedback, recurrence gates, adaptive thresholds.
3	2026-04-23 – 2026-05-19	RCA quality close-out, hallucination firewall epic added on 2026-04-29 after 0b215ef3, 20-day investigation window 04-30 – 05-19.
4	TBD	Ranked candidate list: incident correlator, Tier 3 agentic gather, curated RAG, pod-rule fixes, R-1..R-5 resilience tier. GPU migration dropped on cost grounds.

Table C.1. Sprint chronology, condensed.

References

Bibliography

- [1] B. Beyer, C. Jones, J. Petoff, N. R. Murphy (eds.), *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly, 2016.
- [2] B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, S. Thorne (eds.), *The Site Reliability Workbook: Practical Ways to Implement SRE*, O'Reilly, 2018.
- [3] P. He, J. Zhu, Z. Zheng, M. R. Lyu, *Drain: An Online Log Parsing Approach with Fixed Depth Tree*, IEEE International Conference on Web Services (ICWS), 2017.
- [4] OpenTelemetry Project, *OpenTelemetry Specification*, version 1.x, <https://opentelemetry.io/docs/specs/otel/>.
- [5] J. Volz et al., *Prometheus: Up & Running*, O'Reilly, 2018, and <https://prometheus.io/docs/>.
- [6] Grafana Labs, *Grafana Loki documentation*, <https://grafana.com/docs/loki/latest/>.
- [7] Y. Shkuro, *Mastering Distributed Tracing*, Packt, 2019, and <https://www.jaegertracing.io/docs/>.
- [8] Anthropic, *Model Context Protocol — specification*, <https://modelcontextprotocol.io/>.
- [9] Qwen Team, Alibaba Group, *Qwen2.5 Technical Report*, arXiv:2412.15115, 2024.
- [10] Ollama, *Run large language models locally*, <https://ollama.com/>.
- [11] Rancher, *k3s — Lightweight Kubernetes*, <https://k3s.io/>.
- [12] HashiCorp, *Terraform documentation*, <https://developer.hashicorp.com/terraform/docs>.
- [13] Red Hat, *Ansible documentation*, <https://docs.ansible.com/>.
- [14] Tailscale Inc., *Tailscale documentation*, <https://tailscale.com/kb/>.